C# Programming for Indian Students — Chapter 8: Arrays

Nov 4, 2026 · @Vikash

8.1 What you will learn

In Chapter 7 your programs learned to organise their work into methods. In this chapter they learn to hold **many values at once**. Think of a class teacher with 60 students: storing their marks in 60 separate variables (`marks1` , `marks2` , … `marks60`) would be impossible to manage, and a loop could not visit them. An **array** keeps all 60 marks under **one name**, and each mark is reached by its position number. By the end of this chapter you will be able to:

- create an array, read and change its elements by **index**, and find its **length**;
- fill an array from the keyboard and print it with `for` and `foreach` ;
- find the total, average, highest and lowest value of an array;
- search an array for a value;
- pass an array to a method, return an array from a method, and explain why a method can change the caller's array;
- sort an array yourself with **bubble sort**, and with the built-in `Array.Sort` ;
- store a table of values, such as the marks of several students in several subjects, in a **two-dimensional array**;
- read the compiler and run-time messages for the most common array mistakes;
- build a Class Result Sheet that uses everything above.

You have already met arrays. In Chapters 6 and 7 we used small lists like `int[] marks = { 91, 78, 64, 45, 32 };` with `foreach` , and promised to explain them later. This is that chapter. As before, every program was compiled and run on the .NET 10 SDK with the sample input shown.

8.2 Your first array

An array is a **fixed-size row of boxes, all of the same type**, kept under one name. Each box is called an **element**, and each element has a position number called its **index**.

```
int[] marks = new int[5];
```

Vikash Kumar

- `int[]` is the type: *an array of ints*. The square brackets come **after the type**, not after the name.
- `new int[5]` creates the array with **5 elements**. The size is fixed: an array cannot grow or shrink later.
- The index starts at **0**. Five elements have the indexes 0, 1, 2, 3 and 4. The last index is always `Length - 1`.
- `marks[2]` means *the element at index 2*, which is the **third** box. You can read it (`Console.WriteLine(marks[2])`) or change it (`marks[2] = 70;`).
- `marks.Length` gives the number of elements. It is a property, so it has no brackets.

Index	0	1	2	3	4
`marks[index]`	78	92	65	88	71

If you already know the values, an **array initializer** creates and fills the array in one line: `string[] cities = { "Delhi", "Mumbai", "Chennai", "Kolkata" };`. The compiler counts the values and sets the length to 4.

Program 8.1: Your first array

```
int[] marks = new int[5];
marks[0] = 78;
marks[1] = 92;
marks[2] = 65;
marks[3] = 88;
marks[4] = 71;

Console.WriteLine($"First mark  : {marks[0]}");
Console.WriteLine($"Third mark  : {marks[2]}");
Console.WriteLine($"Last mark   : {marks[marks.Length - 1]}");
Console.WriteLine($"Length      : {marks.Length}");

marks[2] = 70;
Console.WriteLine($"Third mark after correction: {marks[2]}");

string[] cities = { "Delhi", "Mumbai", "Chennai", "Kolkata" };
Console.WriteLine($"{cities.Length} cities, the first is {cities[0]}");

double[] prices = new double[3];
bool[] present = new bool[3];
string[] names = new string[2];
Console.WriteLine($"Default values: {prices[0]}, {present[0]}, {names[0] ==
null}");
```

Vikash Kumar

Output

```
First mark  : 78
Third mark  : 65
Last mark   : 71
Length      : 5
Third mark after correction: 70
4 cities, the first is Delhi
Default values: 0, False, True
```

What to notice.

- `marks[marks.Length - 1]` always gives the last element, whatever the size of the array. This is safer than writing `marks[4]`.
- `new` fills every element with a **default value**: `0` for numbers, `false` for `bool`, and `null` (nothing at all) for `string`. That is why `names[0] == null` prints `True`. A `null` string has no text yet; using it as text causes an error (section 8.9).
- An array of 60 marks is created exactly the same way: `int[] marks = new int[60];`. One line instead of 60 variables.

8.3 Filling and printing arrays with loops

Arrays and loops are made for each other. The index runs from `0` to `Length - 1`, which is exactly what a `for` loop counts. The pattern below visits every element and is the most common loop you will ever write:

```
for (int i = 0; i < marks.Length; i++)
{
    // work with marks[i]
}
```

Note the `<`, not `<=`. With `<=` the loop would ask for `marks[Length]`, one box past the end (section 8.9). The size of the array can also come from the user: `new int[count]` creates exactly as many boxes as were asked for.

Program 8.2: Filling and printing an array

```
Console.Write("How many students? ");
int count = int.Parse(Console.ReadLine() ?? "");
int[] marks = new int[count];

for (int i = 0; i < marks.Length; i++)
{
    Console.Write($"Marks of student {i + 1}: ");
```

Vikash Kumar

```
    marks[i] = int.Parse(Console.ReadLine() ?? "");
}

Console.WriteLine("Using for (position and value):");
for (int i = 0; i < marks.Length; i++)
{
    Console.WriteLine($"  marks[{i}] = {marks[i]}");
}

Console.Write("Using foreach (values only): ");
foreach (int m in marks)
{
    Console.Write(m + " ");
}
Console.WriteLine();

Console.Write("In reverse order: ");
for (int i = marks.Length - 1; i >= 0; i--)
{
    Console.Write(marks[i] + " ");
}
Console.WriteLine();
```

Output (typed input: `4`, then `78`, `92`, `65`, `88`)

```
How many students? 4
Marks of student 1: 78
Marks of student 2: 92
Marks of student 3: 65
Marks of student 4: 88
Using for (position and value):
  marks[0] = 78
  marks[1] = 92
  marks[2] = 65
  marks[3] = 88
Using foreach (values only): 78 92 65 88
In reverse order: 88 65 92 78
```

for or foreach?

You need to	Use	Why
Fill or change the elements	`for`	`marks[i] = ...` needs the index. The `foreach` variable is read-only (CS1656, Chapter 6).

Vikash Kumar

You need to	Use	Why
Know the position (student 1, 2, ...)	`for`	Only `for` has `i`.
Go backwards or skip elements	`for`	Start at `Length - 1` and use `i--`, or step with `i += 2`.
Only read every value, start to end	`foreach`	Shorter, and it can never go past the end.

People count from 1 and arrays count from 0, so the prompt shows `i + 1` while the program stores into `marks[i]`. Keep the two apart in your head: **the user sees student 1, the array uses index 0**.

8.4 Total, average, highest and lowest

Once the values are in an array, one loop can answer many questions about them. The two patterns to learn are the **running total** from Chapter 6 and the **best so far**:

- **Running total:** start with `total = 0` and add every element.
- **Best so far:** start with the **first element** (`highest = marks[0]`), then compare every element with it and keep the bigger one. Starting with `0` would be wrong for the lowest mark, because no mark is smaller than 0.

Program 8.3: Total, average, highest and lowest

```
int[] marks = { 78, 92, 65, 88, 71, 54, 95, 60 };

int total = 0;
int highest = marks[0];
int lowest = marks[0];

foreach (int m in marks)
{
    total += m;
    if (m > highest)
    {
        highest = m;
    }
    if (m < lowest)
    {
        lowest = m;
    }
}
```

Vikash Kumar

```
double average = (double)total / marks.Length;

int aboveAverage = 0;
foreach (int m in marks)
{
    if (m > average)
    {
        aboveAverage++;
    }
}

Console.WriteLine($"Marks         : {string.Join(", ", marks)}");
Console.WriteLine($"Total         : {total}");
Console.WriteLine($"Average       : {average:F2}");
Console.WriteLine($"Highest       : {highest}");
Console.WriteLine($"Lowest        : {lowest}");
Console.WriteLine($"Above average : {aboveAverage} students");
```

Output

```
Marks         : 78, 92, 65, 88, 71, 54, 95, 60
Total         : 603
Average       : 75.38
Highest       : 95
Lowest        : 54
Above average : 4 students
```

What to notice.

- `(double)total / marks.Length` turns the total into a `double` before dividing. Without the cast, `603 / 8` is integer division and gives `75` (Chapter 4).
- Counting the students above the average needs a **second loop**: the average is known only after the first loop has finished. This is something you could not do without an array, because the values would be gone after reading them once.
- `string.Join(", ", marks)` is a ready-made method that joins all the elements into one string with `", "` between them. It is the quickest way to print a whole array. `Console.WriteLine(marks)` would print only `System.Int32[]`, the name of the type.

8.5 Searching an array

To find a value, look at the elements one by one from the start until you find it or reach the end. This is called **linear search**. Two arrays of the same length can work as a pair: element `i` of `rollNumbers` and element `i` of `names` describe the same student. Such arrays are

Vikash Kumar

called **parallel arrays**.

The search remembers **where** it found the value. The variable `position` starts at `-1`, a value that can never be a real index, and means *not found yet*.

Program 8.4: Linear search

```
int[] rollNumbers = { 101, 104, 107, 110, 113, 116 };
string[] names = { "Aarav", "Diya", "Kabir", "Meera", "Rohan", "Sneha" };

Console.Write("Enter a roll number to search: ");
int target = int.Parse(Console.ReadLine() ?? "");

int position = -1;
for (int i = 0; i < rollNumbers.Length; i++)
{
    if (rollNumbers[i] == target)
    {
        position = i;
        break;
    }
}

if (position >= 0)
{
    Console.WriteLine($"Found at index {position}: {names[position]}");
}
else
{
    Console.WriteLine($"Roll number {target} is not in the class list.");
}
```

Output (two runs: typed `110`, then typed `105`)

```
Enter a roll number to search: 110
Found at index 3: Meera
```

```
Enter a roll number to search: 105
Roll number 105 is not in the class list.
```

How it works.

- For `110` the loop compares 101, 104, 107 and then 110. At index 3 it matches, stores `position = 3`, and `break` ends the loop at once (Chapter 6). There is no need to look at the rest.
- For `105` no element matches, the loop runs to the end, and `position` is still `-1`.

Vikash Kumar

- The index found in one array is used to read the other: `names[3]` is `"Meera"`.
- A linear search of 60 students makes at most 60 comparisons. In section 8.7 you will meet `Array.IndexOf`, which does exactly this search for you.

8.6 Arrays and methods

An array can be a parameter and a return value, just like an `int`. Write the type with brackets in the method's definition: `int Sum(int[] values)` takes an array, and `int[] AddGrace(...)` gives an array back. At the call you pass the array's name without brackets: `Sum(marks)`.

Program 8.5: Arrays as parameters and return values

```
int[] marks = { 78, 92, 65, 88, 71 };

PrintArray("Marks", marks);
Console.WriteLine($"Total      : {Sum(marks)}");
Console.WriteLine($"Highest    : {Max(marks)}");

int[] withGrace = AddGrace(marks, 10);
PrintArray("With grace", withGrace);
PrintArray("Original", marks);

void PrintArray(string label, int[] values)
{
    Console.WriteLine($"{label,-11}: {string.Join(" ", values)}");
}

int Sum(int[] values)
{
    int total = 0;
    foreach (int v in values)
    {
        total += v;
    }
    return total;
}

int Max(int[] values)
{
    int biggest = values[0];
    for (int i = 1; i < values.Length; i++)
    {
        if (values[i] > biggest)
        {
```

Vikash Kumar

```
            biggest = values[i];
        }
    }
    return biggest;
}

int[] AddGrace(int[] values, int grace)
{
    int[] result = new int[values.Length];
    for (int i = 0; i < values.Length; i++)
    {
        result[i] = Math.Min(values[i] + grace, 100);
    }
    return result;
}
```

Output

```
Marks      : 78 92 65 88 71
Total      : 394
Highest    : 92
With grace : 88 100 75 98 81
Original   : 78 92 65 88 71
```

`Sum` and `Max` work for an array of **any** length, so they can be reused for 5 students or 500. `AddGrace` builds a **new** array and returns it; `Math.Min(..., 100)` caps 92 + 10 at 100. The original marks are untouched because `AddGrace` only reads them.

Arrays are references. In Chapter 7 a method received a **copy** of an `int`, so it could not change the caller's variable. An array behaves differently. The array variable does not hold the boxes themselves; it holds a **reference**, the address of the boxes in memory. Passing `marks` to a method copies that address, so the method and the caller reach **the same boxes**.

Program 8.6: A method can change the caller's array

```
int number = 50;
int[] marks = { 50, 60, 70 };

AddFiveToNumber(number);
AddFiveToArray(marks);
Console.WriteLine($"number after the method  : {number}");
Console.WriteLine($"marks after the method   : {string.Join(", ", marks)}");

int[] sameArray = marks;
sameArray[0] = 0;
```

Vikash Kumar

```
Console.WriteLine($"marks after sameArray[0] = 0 : {string.Join(", ",
marks)}");

int[] realCopy = new int[marks.Length];
Array.Copy(marks, realCopy, marks.Length);
realCopy[1] = 0;
Console.WriteLine($"marks after realCopy[1] = 0  : {string.Join(", ",
marks)}");
Console.WriteLine($"realCopy                     : {string.Join(", ",
realCopy)}");

void AddFiveToNumber(int n)
{
    n += 5;
}

void AddFiveToArray(int[] values)
{
    for (int i = 0; i < values.Length; i++)
    {
        values[i] += 5;
    }
}
```

Output

```
number after the method  : 50
marks after the method   : 55, 65, 75
marks after sameArray[0] = 0 : 0, 65, 75
marks after realCopy[1] = 0  : 0, 65, 75
realCopy                     : 0, 0, 75
```

Explanation.

- `AddFiveToNumber` changes only its copy, so `number` stays 50 (pass by value, section 7.6).
- `AddFiveToArray` receives the address of the same three boxes, so the caller sees 55, 65, 75. No `ref` is needed.
- `int[] sameArray = marks;` copies only the address. Both names point at one array, so `sameArray[0] = 0` also changes `marks[0]`.
- To get an independent copy, create a new array and copy the elements into it. `Array.Copy(source, destination, length)` does that. Changing `realCopy` leaves `marks` alone.

Vikash Kumar

Kind of variable	What is copied when you pass it or assign it	Can a method change the caller's data?
`int`, `double`, `bool`, `decimal` (value types)	The value itself	No (unless `ref` or `out`)
An array such as `int[]` (a reference type)	The reference (address) only	Yes, the elements

8.7 Sorting, and the built-in Array methods

Bubble sort is the classic first sorting method, and a favourite exam question. Compare each pair of neighbours and **swap** them if they are in the wrong order. After one full pass the largest value has "bubbled" to the end. Repeat the passes; each pass can stop one place earlier, because the end is already sorted.

Program 8.7: Bubble sort, printing every pass

```
int[] marks = { 64, 25, 12, 22, 11 };
Console.WriteLine($"Before : {string.Join(" ", marks)}");

for (int pass = 1; pass < marks.Length; pass++)
{
    for (int i = 0; i < marks.Length - pass; i++)
    {
        if (marks[i] > marks[i + 1])
        {
            (marks[i], marks[i + 1]) = (marks[i + 1], marks[i]);
        }
    }
    Console.WriteLine($"Pass {pass} : {string.Join(" ", marks)}");
}

Console.WriteLine($"After  : {string.Join(" ", marks)}");
```

Output

```
Before : 64 25 12 22 11
Pass 1 : 25 12 22 11 64
Pass 2 : 12 22 11 25 64
Pass 3 : 12 11 22 25 64
Pass 4 : 11 12 22 25 64
After  : 11 12 22 25 64
```

Vikash Kumar

How it works.

- In pass 1, 64 is compared with each neighbour in turn and swapped every time, so it ends at the last place.
- 5 elements need at most 4 passes (`pass < marks.Length`). Pass 1 compares 4 pairs, pass 2 compares 3, and so on: `i < marks.Length - pass`.
- `(marks[i], marks[i + 1]) = (marks[i + 1], marks[i]);` swaps two elements in one line, the same trick as Question 12 of Chapter 7.
- `marks[i + 1]` is safe because `i` stops one place before the last element.

Built-in Array methods. In real programs you do not write your own sort. The `Array` class gives you ready-made, fast methods:

Method	What it does
`Array.Sort(arr)`	Sorts the array in place, smallest first (A to Z for strings)
`Array.Reverse(arr)`	Reverses the order of the elements in place
`Array.IndexOf(arr, value)`	Returns the index of the first match, or `-1` if the value is not there
`Array.Copy(source, dest, length)`	Copies `length` elements from one array into another

Program 8.8: Array.Sort, Array.Reverse and Array.IndexOf

Vikash Kumar

```
int[] scores = { 45, 92, 67, 81, 58, 73 };
string[] states = { "Kerala", "Assam", "Punjab", "Goa", "Bihar" };

int[] sorted = new int[scores.Length];
Array.Copy(scores, sorted, scores.Length);

Array.Sort(sorted);
Console.WriteLine($"Sorted, smallest first : {string.Join(", ", sorted)}");
Array.Reverse(sorted);
Console.WriteLine($"Sorted, largest first  : {string.Join(", ", sorted)}");
Console.WriteLine($"Original is unchanged  : {string.Join(", ", scores)}");

Array.Sort(states);
Console.WriteLine($"States A to Z          : {string.Join(", ", states)}");

Console.WriteLine($"Index of 81            : {Array.IndexOf(scores, 81)}");
Console.WriteLine($"Index of 100           : {Array.IndexOf(scores, 100)}");
Console.WriteLine($"Is Goa in the list?    : {Array.IndexOf(states, "Goa") >=
0}");
```

Output

```
Sorted, smallest first : 45, 58, 67, 73, 81, 92
Sorted, largest first  : 92, 81, 73, 67, 58, 45
Original is unchanged  : 45, 92, 67, 81, 58, 73
States A to Z          : Assam, Bihar, Goa, Kerala, Punjab
Index of 81            : 3
Index of 100           : -1
Is Goa in the list?    : True
```

`Array.Sort` and `Array.Reverse` change the array they are given. To keep the original order, sort a copy, as this program does. Learn bubble sort to understand **how** sorting works; use `Array.Sort` when you only need the result.

8.8 Two-dimensional arrays

A mark sheet, a timetable and a cinema seating chart all have **rows and columns**. A **two-dimensional array** stores such a table under one name. Each element needs two indexes: the row and the column.

```
int[,] marks = new int[3, 4];   // 3 rows (students), 4 columns (subjects)
```

- `int[,]`: the comma inside the brackets means *two dimensions*.

Vikash Kumar

- `marks[1, 2]` is the element in **row 1, column 2**. Both indexes start at 0, so this is the second student's third subject.
- `marks.GetLength(0)` gives the number of rows and `marks.GetLength(1)` the number of columns. `marks.Length` gives the total number of cells (3 x 4 = 12).
- An initializer writes one `{ ... }` per row, separated by commas.

	Column 0 (Maths)	Column 1 (Phys)	Column 2 (Chem)	Column 3 (Eng)
Row 0 (Aarav)	85	78	92	70
Row 1 (Diya)	90	88	**76** = `marks[1, 2]`	84
Row 2 (Kabir)	67	72	80	91

A table is processed with the **nested loops** of Chapter 6: the outer loop walks the rows and the inner loop walks the columns of one row.

Program 8.9: A marks table in a two-dimensional array

```
string[] students = { "Aarav", "Diya", "Kabir" };
string[] subjects = { "Maths", "Phys", "Chem", "Eng" };

int[,] marks =
{
    { 85, 78, 92, 70 },
    { 90, 88, 76, 84 },
    { 67, 72, 80, 91 }
};

int rows = marks.GetLength(0);
int cols = marks.GetLength(1);
Console.WriteLine($"Rows = {rows}, columns = {cols}, cells =
{marks.Length}");
Console.WriteLine($"Diya's Chemistry marks = {marks[1, 2]}");
Console.WriteLine();

Console.Write($"{"Student",-8}");
foreach (string subject in subjects)
{
    Console.Write($"{subject,6}");
}
Console.WriteLine($"{"Total",7}");
```

Vikash Kumar

```
for (int r = 0; r < rows; r++)
{
    Console.Write($"{students[r],-8}");
    int total = 0;
    for (int c = 0; c < cols; c++)
    {
        Console.Write($"{marks[r, c],6}");
        total += marks[r, c];
    }
    Console.WriteLine($"{total,7}");
}

Console.Write($"{"Average",-8}");
for (int c = 0; c < cols; c++)
{
    int sum = 0;
    for (int r = 0; r < rows; r++)
    {
        sum += marks[r, c];
    }
    Console.Write($"{(double)sum / rows,6:F1}");
}
Console.WriteLine();
```

Output

```
Rows = 3, columns = 4, cells = 12
Diya's Chemistry marks = 76

Student  Maths  Phys  Chem   Eng  Total
Aarav       85    78    92    70    325
Diya        90    88    76    84    338
Kabir       67    72    80    91    310
Average   80.7  79.3  82.7  81.7
```

What to notice.

- **Row totals:** the outer loop is over rows, so `total` is reset for every student and printed at the end of the row.
- **Column averages:** the loops are swapped. The outer loop is over columns (subjects) and the inner loop adds that column for every student.
- `{marks[r, c],6}` prints each number right-aligned in 6 characters (Chapter 6), so the columns line up. `{"Student",-8}` is left-aligned in 8.

Vikash Kumar

- The names and subjects live in their own 1D arrays. Row `r` of `marks` belongs to `students[r]`, and column `c` to `subjects[c]`: parallel arrays again.

8.9 Dry run and common errors

Dry run. For an array program, keep one column for the index, one for the element and one for each variable that changes. Take Program 8.10:

```
int[] a = { 3, 8, 2, 9, 4 };
int result = 0;

for (int i = 0; i < a.Length; i++)
{
    if (a[i] % 2 == 0)
    {
        result += a[i];
    }
    else
    {
        result -= a[i];
    }
}

Console.WriteLine($"result = {result}");
```

i	a[i]	Even?	Work done	result
start				0
0	3	No	0 - 3	-3
1	8	Yes	-3 + 8	5
2	2	Yes	5 + 2	7
3	9	No	7 - 9	-2
4	4	Yes	-2 + 4	2
5			`5 < 5` is False, the loop ends	2

Output

```
result = 2
```

Vikash Kumar

Common compile-time errors. The messages below are the real build output of the .NET 10 SDK.

Mistake	What you see	The fix
No size and no values: `int[] a = new int[];`	`error CS1586: Array creation must have array size or array initializer`	Give a size (`new int[5]`) or the values (`{ 1, 2, 3 }`)
Brackets after the name, as in C or Java: `int a[] = new int[5];`	`error CS0650: Bad array declarator: To declare a managed array the rank specifier precedes the variable's identifier.` (shortened)	Write the brackets after the type: `int[] a`
Size and number of values differ: `new int[3] { 1, 2 }`	`error CS0847: An array initializer of length '3' is expected`	Give exactly 3 values, or leave out the size
Brackets after Length: `a.Length()`	`error CS1955: Non-invocable member 'Array.Length' cannot be used like a method.`	`a.Length` , without brackets
Two indexes on a 1D array: `a[1, 0]`	`error CS0022: Wrong number of indices inside []; expected 1`	One index for `int[]` , two for `int[,]`
Using an array that was never created: `int[] a; a[0] = 1;`	`error CS0165: Use of unassigned local variable 'a'`	Create it first: `int[] a = new int[5];`
Storing the wrong type: `int x = names[0];` (a `string[]`)	`error CS0029: Cannot implicitly convert type 'string' to 'int'`	Convert with `int.Parse` , or use the right type

Run-time errors. These compile without any message and stop the program when it runs.

1. **Index out of range, the most common array mistake.** The loop below uses `<=` , so it asks for `marks[3]` in an array whose last index is 2:

```
int[] marks = { 78, 92, 65 };
for (int i = 0; i <= marks.Length; i++)
{
    Console.WriteLine(marks[i]);
}
```

Vikash Kumar

```
78
92
65
Unhandled exception. System.IndexOutOfRangeException: Index was outside
the bounds of the array.
```

The fix is `i < marks.Length`. The same error appears for a negative index, or for an index that came from the user and was never checked.

2. **An element that is still null.** `new string[3]` creates three empty boxes (section 8.2). Using one as text, `Console.WriteLine(names[0].Length);`, stops the program with:

```
Unhandled exception. System.NullReferenceException: Object reference not
set to an instance of an object.
```

Fill every element before you use it, as Program 8.2 does in its first loop.

8.10 Mini project: Class Result Sheet

In Chapter 5 you calculated the result of **one** student. Now build the result sheet for a **whole class**: names in a `string[]`, marks in a two-dimensional `int[,]`, totals in an `int[]`, and every job in its own method, as in Chapter 7.

Requirements

1. Ask for the number of students (1 to 10). Then, for each student, the name and the marks in Maths, Science and English (0 to 100). Ask again on a wrong entry or an empty name.
2. Print a result sheet: marks, total, percentage, grade and result for every student.
3. A student **passes** only if every subject has at least 35 marks.
4. Print the class average of each subject and the name of the class topper.

Arrays used

Array	Type	Holds
`subjects`	`string[]`	The three subject names
`names`	`string[]`	One name per student
`marks`	`int[,]`	Row = student, column = subject
`totals`	`int[]`	One total per student, filled while reading

Methods used

Vikash Kumar

Method	Job	Returns
`ReadNumber(prompt, min, max)`	Ask until a whole number in the range is typed	`int`
`ReadName(prompt)`	Ask until a name that is not empty is typed	`string`
`PrintResultSheet(...)`	Print the table with total, percentage, grade, result	Nothing
`PrintSubjectAverages(subjects, marks)`	Print the average of every column	Nothing
`PassedAll(marks, student)`	Check one row: every subject at least 35?	`bool`
`IndexOfMax(values)`	Find the index of the largest total	`int`
`GetGrade(percentage)`	Turn a percentage into a grade	`string`

Program 8.11: Class Result Sheet

```
string[] subjects = { "Maths", "Science", "English" };

int count = ReadNumber("Number of students (1-10): ", 1, 10);
string[] names = new string[count];
int[,] marks = new int[count, subjects.Length];
int[] totals = new int[count];

for (int s = 0; s < count; s++)
{
    Console.WriteLine();
    names[s] = ReadName($"Name of student {s + 1}: ");
    for (int j = 0; j < subjects.Length; j++)
    {
        marks[s, j] = ReadNumber($"  {subjects[j]} marks (0-100): ", 0, 100);
        totals[s] += marks[s, j];
    }
}

PrintResultSheet(names, subjects, marks, totals);
PrintSubjectAverages(subjects, marks);

int topper = IndexOfMax(totals);
Console.WriteLine($"Class topper: {names[topper]} with {totals[topper]}
marks");
```

Vikash Kumar

```
int ReadNumber(string prompt, int min, int max)
{
    while (true)
    {
        Console.Write(prompt);
        if (int.TryParse(Console.ReadLine(), out int value) && value >= min
&& value <= max)
        {
            return value;
        }
        Console.WriteLine($"Please type a whole number from {min} to
{max}.");
    }
}

string ReadName(string prompt)
{
    while (true)
    {
        Console.Write(prompt);
        string name = (Console.ReadLine() ?? "").Trim();
        if (name.Length > 0)
        {
            return name;
        }
        Console.WriteLine("The name cannot be empty.");
    }
}

void PrintResultSheet(string[] names, string[] subjects, int[,] marks, int[]
totals)
{
    Console.WriteLine();
    Console.WriteLine("=============== Class Result Sheet ===============");
    Console.Write($"{"Name",-8}");
    foreach (string subject in subjects)
    {
        Console.Write($"{subject,8}");
    }
    Console.WriteLine($"{"Total",7}{"%",8}{"Grade",7}  Result");

    for (int s = 0; s < names.Length; s++)
    {
        Console.Write($"{names[s],-8}");
        for (int j = 0; j < subjects.Length; j++)
```

Vikash Kumar

```
        {
            Console.Write($"{marks[s, j],8}");
        }
        double percentage = totals[s] * 100.0 / (subjects.Length * 100);
        string result = PassedAll(marks, s) ? "Pass" : "Fail";
        Console.WriteLine($"{totals[s],7}{percentage,8:F2}
{GetGrade(percentage),7}  {result}");
    }
}

void PrintSubjectAverages(string[] subjects, int[,] marks)
{
    Console.Write($"{"Average",-8}");
    for (int j = 0; j < subjects.Length; j++)
    {
        int sum = 0;
        for (int s = 0; s < marks.GetLength(0); s++)
        {
            sum += marks[s, j];
        }
        Console.Write($"{(double)sum / marks.GetLength(0),8:F2}");
    }
    Console.WriteLine();
}

bool PassedAll(int[,] marks, int student)
{
    for (int j = 0; j < marks.GetLength(1); j++)
    {
        if (marks[student, j] < 35)
        {
            return false;
        }
    }
    return true;
}

int IndexOfMax(int[] values)
{
    int best = 0;
    for (int i = 1; i < values.Length; i++)
    {
        if (values[i] > values[best])
        {
            best = i;
        }
```

Vikash Kumar

```
    }
    return best;
}

string GetGrade(double percentage)
{
    return percentage switch
    {
        >= 90 => "A+",
        >= 75 => "A",
        >= 60 => "B",
        >= 40 => "C",
        _ => "Fail"
    };
}
```

Output (typed input: `0`, `3`; `Aarav`, `88`, `92`, `79`; an empty line, `Diya`, `101`, `95`, `90`, `84`; `Kabir`, `45`, `30`, `62`)

```
Number of students (1-10): 0
Please type a whole number from 1 to 10.
Number of students (1-10): 3

Name of student 1: Aarav
  Maths marks (0-100): 88
  Science marks (0-100): 92
  English marks (0-100): 79

Name of student 2:
The name cannot be empty.
Name of student 2: Diya
  Maths marks (0-100): 101
Please type a whole number from 0 to 100.
  Maths marks (0-100): 95
  Science marks (0-100): 90
  English marks (0-100): 84

Name of student 3: Kabir
  Maths marks (0-100): 45
  Science marks (0-100): 30
  English marks (0-100): 62

=============== Class Result Sheet ===============
Name       Maths Science English  Total       %  Grade  Result
Aarav         88      92      79    259   86.33      A  Pass
```

Vikash Kumar

```
Diya          95      90      84    269   89.67      A  Pass
Kabir         45      30      62    137   45.67      C  Fail
Average    76.00   70.67   75.00
Class topper: Diya with 269 marks
```

Design decisions.

- The arrays are created **after** the count is known: `new int[count, subjects.Length]`. Nothing is hard-coded to 3 students or 3 subjects, so adding a fourth subject only means adding a name to `subjects`.
- `ReadNumber` serves both the student count and every mark, because the range is a parameter. This is the payoff of methods from Chapter 7.
- `totals[s] += marks[s, j];` works because `new int[count]` starts every total at 0 (section 8.2).
- Kabir's percentage earns grade C, but his Science mark is 30, so `PassedAll` returns `false` and the result is Fail. The grade and the result are separate decisions, made by separate methods.
- `IndexOfMax` returns the **index** of the topper, not the total. With the index, the program can print the name from `names` and the total from `totals`.
- Challenge: print the students in order of rank, topper first. Hint: sort a copy of the indexes by their totals with bubble sort.

8.11 Practice questions with answers

Try every question yourself before reading the answer. The solution programs are in the `Solutions` folder of the chapter in the code repository.

Q1. What is an array? Why is it better than separate variables for the marks of 60 students? An array is a fixed-size collection of elements of the same type under one name, each reached by an index. With an array, one loop can read, total or print all 60 marks; with 60 separate variables every statement would have to be written 60 times.

Q2. An array has 8 elements. What are its first and last indexes? What happens if you use index 8? The first index is 0 and the last is 7 (`Length - 1`). Index 8 compiles, but stops the program at run time with `IndexOutOfRangeException`.

Q3. When must you use `for` instead of `foreach` with an array? When you change the elements (`arr[i] = ...`), when you need the position, or when you go backwards or skip elements. `foreach` only reads each value from start to end.

Q4. A method receives an `int` and an `int[]` and changes both. Which change does the caller see, and why? Only the change to the array. An `int` is copied, but for an array only the reference is copied, so the method and the caller use the same elements.

Vikash Kumar

Q5. What do `arr.Length`, `table.GetLength(0)` and `table.GetLength(1)` give for `int[,] table = new int[4, 6];`? `table.Length` is 24 (all cells), `GetLength(0)` is 4 (rows) and `GetLength(1)` is 6 (columns). For a 1D array `arr`, `arr.Length` is the number of elements.

Q6. Predict the output.

```
int[] a = { 10, 20, 30, 40 };
Console.WriteLine(a[1] + a[a.Length - 1]);
Console.WriteLine(a.Length);
a[2] = a[0] + 5;
foreach (int x in a)
{
    Console.Write(x + " ");
}
Console.WriteLine();
```

```
60
4
10 20 15 40
```

`a[1]` is 20 and `a[3]` is 40. Then `a[2]` becomes 10 + 5 = 15.

Q7. Predict the output.

```
int[] p = { 1, 2, 3 };
int[] q = p;
q[0] = 100;
Console.WriteLine(p[0]);
Change(p);
Console.WriteLine(string.Join(",", p));

void Change(int[] arr)
{
    arr[2] = arr[0] + arr[1];
}
```

```
100
100,2,102
```

`q = p` copies only the reference, so `q[0] = 100` changes `p` too. `Change` works on the same array: 100 + 2 = 102.

Q8. Predict the output.

Vikash Kumar

```
int[,] g = { { 1, 2, 3 }, { 4, 5, 6 } };
int sum = 0;
for (int r = 0; r < g.GetLength(0); r++)
{
    sum += g[r, r];
}
Console.WriteLine(sum);
Console.WriteLine(g.GetLength(1));
Console.WriteLine(g[1, 0] * g[0, 2]);
```

```
6
3
12
```

The loop runs for r = 0 and 1 and adds `g[0, 0]` + `g[1, 1]` = 1 + 5 = 6. There are 3 columns. `g[1, 0]` is 4 and `g[0, 2]` is 3.

Q9. Read 6 numbers into an array and reverse the array in place, without `Array.Reverse`.

```
int[] numbers = new int[6];
for (int i = 0; i < numbers.Length; i++)
{
    Console.Write($"Number {i + 1}: ");
    numbers[i] = int.Parse(Console.ReadLine() ?? "");
}

int left = 0;
int right = numbers.Length - 1;
while (left < right)
{
    (numbers[left], numbers[right]) = (numbers[right], numbers[left]);
    left++;
    right--;
}

Console.WriteLine($"Reversed: {string.Join(" ", numbers)}");
```

```
Number 1: 4
Number 2: 8
Number 3: 15
Number 4: 16
Number 5: 23
Number 6: 42
Reversed: 42 23 16 15 8 4
```

Vikash Kumar

Swap the first and last elements, then move both ends one step inwards. The loop stops when they meet in the middle, after 3 swaps.

Q10. Find the second largest value in an array. The largest may appear twice.

```
int[] marks = { 72, 95, 88, 95, 64, 81 };

int largest = int.MinValue;
int second = int.MinValue;

foreach (int m in marks)
{
    if (m > largest)
    {
        second = largest;
        largest = m;
    }
    else if (m > second && m < largest)
    {
        second = m;
    }
}

Console.WriteLine($"Marks          : {string.Join(", ", marks)}");
Console.WriteLine($"Largest        : {largest}");
Console.WriteLine($"Second largest : {second}");
```

```
Marks          : 72, 95, 88, 95, 64, 81
Largest        : 95
Second largest : 88
```

When a new largest value arrives, the old largest moves down to second place. `m < largest` makes sure the second 95 is not counted as the second largest.

Q11. A canteen survey collected ratings from 1 to 5. Count how many of each rating were given, using an array as counters.

Vikash Kumar

```
int[] ratings = { 5, 3, 4, 5, 2, 5, 4, 3, 5, 1, 4, 5 };
int[] count = new int[6];

foreach (int r in ratings)
{
    count[r]++;
}

Console.WriteLine($"Canteen survey: {ratings.Length} ratings");
for (int star = 5; star >= 1; star--)
{
    Console.WriteLine($"{star} star: {new string('*', count[star]),-6}
({count[star]})");
}
```

```
Canteen survey: 12 ratings
5 star: *****  (5)
4 star: ***    (3)
3 star: **     (2)
2 star: *      (1)
1 star: *      (1)
```

The rating itself is used as the index: `count[r]++` adds 1 to the counter for that rating. The array has 6 elements so that index 5 exists (index 0 is not used). `new string('*', n)` makes a string of n stars.

Q12. Add two 2 x 3 matrices.

```
int[,] a = { { 1, 2, 3 }, { 4, 5, 6 } };
int[,] b = { { 9, 8, 7 }, { 6, 5, 4 } };
int[,] sum = new int[2, 3];

for (int r = 0; r < a.GetLength(0); r++)
{
    for (int c = 0; c < a.GetLength(1); c++)
    {
        sum[r, c] = a[r, c] + b[r, c];
    }
}

Console.WriteLine("A + B =");
for (int r = 0; r < sum.GetLength(0); r++)
{
    for (int c = 0; c < sum.GetLength(1); c++)
    {
```

Vikash Kumar

```
        Console.Write($"{sum[r, c],4}");
    }
    Console.WriteLine();
}
```

```
A + B =
  10  10  10
  10  10  10
```

Matrices are added cell by cell, so both matrices must have the same number of rows and columns.

Q13. Store a weekly class timetable (5 days, 4 periods) in a `string[,]`, print it, and count the C# periods on a day the user types.

```
string[] days = { "Mon", "Tue", "Wed", "Thu", "Fri" };
string[,] timetable =
{
    { "Maths", "Physics", "C#", "English" },
    { "C#", "Maths", "Chemistry", "Lab" },
    { "Physics", "English", "Maths", "C#" },
    { "Chemistry", "C#", "Lab", "Maths" },
    { "English", "Maths", "C#", "Sports" }
};

Console.Write($"{"Day",-5}");
for (int p = 0; p < timetable.GetLength(1); p++)
{
    Console.Write($"{"Period " + (p + 1),-11}");
}
Console.WriteLine();

for (int d = 0; d < days.Length; d++)
{
    Console.Write($"{days[d],-5}");
    for (int p = 0; p < timetable.GetLength(1); p++)
    {
        Console.Write($"{timetable[d, p],-11}");
    }
    Console.WriteLine();
}

Console.Write("Which day (Mon-Fri)? ");
string wanted = Console.ReadLine() ?? "";
int day = Array.IndexOf(days, wanted);
if (day < 0)
```

Vikash Kumar

```
{
    Console.WriteLine("No classes on that day.");
}
else
{
    int csharp = 0;
    for (int p = 0; p < timetable.GetLength(1); p++)
    {
        if (timetable[day, p] == "C#")
        {
            csharp++;
        }
    }
    Console.WriteLine($"{wanted} has {csharp} C# period(s).");
}
```

```
Day  Period 1   Period 2   Period 3   Period 4
Mon  Maths      Physics    C#         English
Tue  C#         Maths      Chemistry  Lab
Wed  Physics    English    Maths      C#
Thu  Chemistry  C#         Lab        Maths
Fri  English    Maths      C#         Sports
Which day (Mon-Fri)? Thu
Thu has 1 C# period(s).
```

`Array.IndexOf(days, wanted)` turns the day name into a row number, and the loop walks along that row only.

Q14. Multiply a 2 x 3 matrix by a 3 x 2 matrix.

```
int[,] a = { { 1, 2, 3 }, { 4, 5, 6 } };
int[,] b = { { 7, 8 }, { 9, 10 }, { 11, 12 } };

int rows = a.GetLength(0);
int inner = a.GetLength(1);
int cols = b.GetLength(1);
int[,] product = new int[rows, cols];

for (int r = 0; r < rows; r++)
{
    for (int c = 0; c < cols; c++)
    {
        int sum = 0;
        for (int k = 0; k < inner; k++)
        {
            sum += a[r, k] * b[k, c];
```

Vikash Kumar

```
        }
        product[r, c] = sum;
    }
}

Console.WriteLine($"A is {rows} x {inner}, B is {inner} x {cols}, A x B is
{rows} x {cols}");
for (int r = 0; r < rows; r++)
{
    for (int c = 0; c < cols; c++)
    {
        Console.Write($"{product[r, c],5}");
    }
    Console.WriteLine();
}
```

```
A is 2 x 3, B is 3 x 2, A x B is 2 x 2
   58   64
  139  154
```

Each cell of the product is row `r` of A multiplied term by term with column `c` of B and added: `product[0, 0]` = 1 x 7 + 2 x 9 + 3 x 11 = 7 + 18 + 33 = 58. This needs three nested loops, and the columns of A must equal the rows of B.

8.12 Multiple choice questions and viva

Multiple choice questions (answers at the end of the list)

1. The index of the first element of an array is:
   - (a) 1 (b) 0 (c) -1 (d) `Length`
2. After `int[] a = new int[4];`, the value of `a.Length` is:
   - (a) 3 (b) 4 (c) 5 (d) 0
3. Every element of `new bool[3]` starts as:
   - (a) `true` (b) `null` (c) `false` (d) 0
4. Which declaration is valid?
   - (a) `int a[] = new int[3];` (b) `int[] a = new int[];` (c) `int[] a = { 1, 2, 3 };` (d) `int[3] a;`
5. What is printed?

Vikash Kumar

```
int[] x = { 2, 4, 6 };
int[] y = x;
y[1] = 0;
Console.WriteLine(x[1]);
```

   - (a) 4 (b) 0 (c) 2 (d) a compile-time error

6. Reading `a[5]` when `a` has 5 elements gives:
   - (a) 0 (b) error CS0165 (c) an `IndexOutOfRangeException` at run time (d) the last element
7. For `int[,] m = new int[3, 5];`, the value of `m.GetLength(1)` is:
   - (a) 3 (b) 5 (c) 15 (d) 2
8. `Array.IndexOf` returns this when the value is not in the array:
   - (a) 0 (b) `null` (c) -1 (d) `Length`
9. At most how many passes does bubble sort need for 6 elements?
   - (a) 6 (b) 5 (c) 36 (d) 3
10. Which loop cannot change the elements of an array through its loop variable?
    - (a) `for` (b) `while` (c) `foreach` (d) `do-while`

Answers: 1 (b), 2 (b), 3 (c), 4 (c), 5 (b; `y` and `x` refer to the same array), 6 (c), 7 (b), 8 (c), 9 (b), 10 (c).

Viva questions (answer in your own words, then compare with the chapter)

1. What is an array, and why do we need it?
2. Why does the index start at 0? What is the last index of an array?
3. What are the default values of the elements of `new int[5]`, `new bool[5]` and `new string[5]`?
4. Why can a method change the elements of an array it receives, but not an `int` it receives?
5. What is the difference between `int[] b = a;` and copying with `Array.Copy`?
6. Explain bubble sort in your own words. Why can each pass stop one place earlier?
7. What is a two-dimensional array? How do you find its number of rows and columns?

Hint for question 5: `int[] b = a;` copies the reference, so `a` and `b` are two names for one array; `Array.Copy` copies the elements into a second array that can be changed on its own.

Vikash Kumar

8.13 Summary and what comes next

- An **array** holds a fixed number of values of one type under one name. `int[] marks = new int[60];` creates 60 elements; `{ 78, 92, 65 }` creates and fills an array in one step.
- Elements are reached by **index**, from `0` to `Length - 1`. `marks.Length` gives the number of elements. An index outside that range stops the program with `IndexOutOfRangeException`.
- `new` fills every element with a default value: `0` for numbers, `false` for `bool`, `null` for `string`.
- Use `for (int i = 0; i < arr.Length; i++)` to fill, change or walk an array by position; use `foreach` to read every value. `string.Join(", ", arr)` prints a whole array.
- The **running total** and **best so far** patterns give the total, average, highest and lowest. **Linear search** looks at each element and remembers the index, with `-1` for *not found*.
- Arrays can be passed to and returned from methods. An array is a **reference**: a method that receives it can change the caller's elements, and `b = a` makes two names for one array. Copy with `Array.Copy` to get an independent array.
- **Bubble sort** swaps neighbours in passes until the array is in order. `Array.Sort`, `Array.Reverse` and `Array.IndexOf` do the same jobs for you.
- A **two-dimensional array** `int[,]` stores a table. `m[r, c]` is one cell; `GetLength(0)` and `GetLength(1)` give the rows and columns; nested loops process it.
- The compiler catches the common declaration mistakes: CS1586 (no size), CS0650 (brackets after the name), CS0847 (wrong number of values), CS1955 (`Length()`), CS0022 (wrong number of indexes) and CS0165 (array never created).

Next chapter: Chapter 9, Strings, looks closely at the type you have used since Chapter 1. You will search and change text with string methods such as `Length`, `ToUpper`, `Contains`, `IndexOf`, `Substring`, `Replace` and `Split`, format output, and check user input such as names, mobile numbers and PIN codes.

Yilash Kumar